

DAY 04

WEEK LECTURE COURSE - DAY 04

SSG 정적 빌드 & 파일 시스템 파서 그리고 마크다운 기술 블로그

HTML 사전 컴파일과 파일 파싱 아키텍처 학습

Target: 대학 1학년 및 취업훈련생 입문 과정

Framework: Next.js 14/15 + Tailwind CSS

Date: 2026 Curriculum Edition

1. 목차 (Table of Contents)

- 학습 목표
- 도입: 뷔페식 배식(SSG) vs 즉석 주문 조리(SSR)
- 개념 개론: SSG의 동작 방식과 Node.js File System (`fs`) 모듈
- 핵심 기술: Frontmatter(머리말) 파싱 및 동적 경로 `generateStaticParams`
- 실습: Mini-Project 4 - 로컬 마크다운 기술 블로그 (`/blog`)
- 실제 사례 연구: 무중단 배포와 정적 웹 호스팅 (Vercel, Netlify)
- 결론 및 활용
- 강사 첨언 가이드 (Instructor Notes)

2. 학습 목표 (Learning Objectives)

1. ****SSG(정적 사이트 생성)****의 개념과 SSR과의 성능적/비용적 차이를 설명할 수 있다.
2. Node.js의 내장 모듈인 `fs` (파일 시스템)와 `path` 를 활용해 서버 환경에서 로컬 파일을 탐색하고 읽을 수 있다.
3. 마크다운(.md) 문서의 ****Frontmatter(YAML 메타데이터)****를 수동으로 분리하고 파싱하는 로직을 이해한다.
4. Next.js App Router의 동적 세그먼트(`[slug]`)와 ****`generateStaticParams`** **를 사용하여 빌드 타임에 정적 경로를 미리 생성하는 구조를 익힌다.

3. 도입: 뷔페식 배식(SSG) vs 즉석 주문 조리(SSR)

어제 우리는 영화 정보를 매번 새로 가져오는 SSR을 실습했습니다. 하지만 블로그 글이나 공지사항처럼 한 번 써두면 거의 바뀌지 않는 콘텐츠를 매번 사용자가 들어올 때마다 데이터베이스에서 조회하고 새로 그리면 서버 자원이 낭비되고 비용도 증가합니다.

💡 직관적인 비유로 이해하는 SSG

- 즉석 주문 조리 (SSR) - "라면 즉석 요리"
 - 손님이 올 때마다 물을 끓이고 라면을 뜯어 새로 조리합니다. 손님은 조리 시간을 기다려야 합니다.
- 뷔페식 배식 (SSG) - "세팅 완료된 급식"

- 손님이 오기 전에 급식판에 국 and 밥을 미리 다 퍼놓습니다(빌드 타임에 HTML 생성). 손님은 오자마자 식판을 집어(로딩 없이 0.01초 만에 화면 표시) 즉시 식사합니다.

Next.js의 ****SSG(Static Site Generation)****는 우리가 프로젝트를 빌드(`npm run build`)하는 시점에 서버에 있는 마크다운 파일이나 DB를 읽어 모든 HTML 페이지를 물리적인 파일로 하드디스크에 미리 저장해 놓는 혁신적인 속도 개선 기법입니다.

4. 개념 개론: Node.js 파일 시스템과 마크다운

마크다운 파일(.md) 상단에는 작성자, 제목, 날짜 같은 데이터를 다음과 같은 규칙으로 적어둡니다. 이를 **Frontmatter**라고 합니다.

```
---
title: "Next.js App Router 가이드"
date: "2026-05-28"
description: "App Router의 기본에 대해 알아봅니다."
---
이 아래부터는 실제 블로그 글의 본문이 들어갑니다...
```

이 마크다운 문자열을 서버에서 줄 단위로 읽어, `---` 로 둘러싸인 윗부분은 메타데이터 객체로 변환하고, 아랫부분은 HTML 본문 태그로 변환하는 작업을 수행할 것입니다.

5. 핵심 기술: `generateStaticParams`

동적 경로 페이지(`app/blog/[slug]/page.tsx`)는 원래 브라우저가 접속할 때마다 동적으로 렌더링됩니다. 하지만 이 페이지가 SSG로 빌드되게 하려면, 빌드 도중에 서버에게 **"여기 내가 작성한 블로그 글의 제목 리스트 (Slug)들이 있으니, 이 경로들에 대해서 미리 HTML을 구워놓으렴"** 하고 알려주어야 합니다.

이 역할을 수행하는 특수 함수가 바로 `generateStaticParams` 입니다.

```
export async function generateStaticParams() {
  // 빌드 타임에 생성할 블로그 경로 목록을 배열로 반환
  return [
    { slug: 'nextjs-intro' },
    { slug: 'tailwind-guide' }
  ];
}
```

6. 실습 - Mini-Project 4: 마크다운 기술 블로그

이 프로젝트에서는 로컬 폴더(`posts/`)에 위치한 실제 마크다운 파일을 Node.js 파일 시스템 모듈로 읽고 파싱하여 아름다운 UI의 블로그 목록 페이지(`/blog`) 및 동적 정적 상세 페이지(`/blog/[slug]`)를 빌드합니다.

1) 가상 마크다운 데이터 폴더 및 파일 구성

Next.js 프로젝트 루트 디렉토리에 `posts` 폴더를 생성하고, 아래 두 개의 마크다운 파일을 생성합니다.

파일 1: `posts/nextjs-architecture.md`

```
---
title: "Next.js App Router 아키텍처 개러닝"
date: "2026-05-28"
description: "서버 컴포넌트와 클라이언트 컴포넌트의 경계를 어떻게 설계할 것인가에 대한"
---
# Next.js App Router 아키텍처 개러닝

Next.js App Router는 리액트 컴포넌트를 서버에서 먼저 렌더링하여 성능을 극대화합니다.

## 핵심 개념 두 가지
1. **서버 컴포넌트**: 기본적으로 동작하며 데이터베이스 직접 조회 및 파일 읽기가 가능합니다.
2. **클라이언트 컴포넌트**: 브라우저 인터랙션을 처리하며 `useState`, `useEffect` 등을 사용합니다.

## 실무 설계 팁
일반적으로 데이터 패칭 및 큰 레이아웃은 **서버 컴포넌트**로 구축하고, 클릭 이벤트나 타이
```

파일 2: `posts/tailwind-workflow.md`

```

---
title: "Tailwind CSS를 활용한 고속 퍼블리싱"
date: "2026-05-27"
description: "CSS 파일 관리가 필요 없는 유틸리티 퍼스트 클래스 디자인 구축 워크플로우."
---

# Tailwind CSS를 활용한 고속 퍼블리싱

Tailwind CSS는 CSS 파일을 따로 작성하지 않고 HTML 태그의 `className`에 즉석 스타일링을

## 주요 장점
- **CSS 파일 비대화 방지**: 프로젝트 규모가 커져도 최종 빌드 CSS 용량은 일정 수준으로 유
- **빠른 스타일링**: 마우스 이동 및 파일 전환 없이 즉석에서 다크모드 및 반응형 웹을 구현

## 반응형 예제
`w-full md:w-1/2 lg:w-1/3`과 같은 한 줄의 유틸리티 선언만으로 디바이스별 반응형 너비를

```

2) 마크다운 파서 및 파일 로더 헬퍼 함수 구현: `lib/blog.ts`

외부 라이브러리(gray-matter, marked 등) 의존성 문제 없이 순수 자바스크립트로 파일 로더와 마크다운 프론트매터 및 일부 기본 태그 파서를 구현합니다. 학생들에게 정밀한 파싱 알고리즘을 교육하기에 훌륭한 교재가 됩니다.

```

import fs from 'fs';
import path from 'path';

export interface Post {
  slug: string;
  title: string;
  date: string;
  description: string;
  content: string;
}

const postsDirectory = path.join(process.cwd(), 'posts');

// 간단한 커스텀 Frontmatter 및 마크다운 파서 함수
function parseMarkdown(fileContent: string, slug: string): Post {
  const lines = fileContent.split('\n');
  const metadata: Record<string, string> = {};
  let contentStartIndex = 0;
  let isFrontmatter = false;
  let frontmatterLines: string[] = [];

  for (let i = 0; i < lines.length; i++) {
    const line = lines[i].trim();
    if (line === '---') {
      if (!isFrontmatter) {
        isFrontmatter = true;
      } else {
        isFrontmatter = false;
        contentStartIndex = i + 1;
        break;
      }
    } else if (isFrontmatter) {
      frontmatterLines.push(line);
    }
  }

  // frontmatter 분석 (key: value)
  frontmatterLines.forEach(line => {
    const colonIndex = line.indexOf(':');
    if (colonIndex !== -1) {
      const key = line.substring(0, colonIndex).trim();
      let value = line.substring(colonIndex + 1).trim();
      // 양끝 따옴표 제거
    }
  });

```

```

    if (value.startsWith('"') && value.endsWith('"')) {
      value = value.substring(1, value.length - 1);
    }
    metadata[key] = value;
  }
});

const rawBody = lines.slice(contentStartIndex).join('\n');

// 초급 교안용 단순 마크다운 → HTML 변환기 구현 (H1, H2, Bold, List)
let parsedContent = rawBody
  .replace(/^#\s+(.)$/gm, '<h1 class="text-3xl font-extrabold text-slate-100 mt-6">$1</h1>')
  .replace(/^#\#\s+(.)$/gm, '<h2 class="text-2xl font-bold text-slate-200 mt-6">$1</h2>')
  .replace(/\*\*(.+?)\*\*/g, '<strong class="font-bold text-teal-400">$1</strong>')
  .replace(/^-\s+(.)$/gm, '<li class="list-disc list-inside text-slate-300 ml-4">$1</li>')
  .replace(/\r\n/g, '\n')
  .split('\n\n')
  .map(para => {
    const trimmed = para.trim();
    if (!trimmed) return '';
    // HTML 태그로 이미 감싸져 있는 헤더나 리스트는 p 태그로 다시 감싸지 않음
    if (trimmed.startsWith('<h1>') || trimmed.startsWith('<h2>') || trimmed.startsWith('<strong>') || trimmed.startsWith('<li>')) {
      return trimmed;
    }
    return `<p class="text-slate-300 leading-relaxed mb-4">${trimmed}</p>`;
  })
  .filter(Boolean)
  .join('\n');

return {
  slug,
  title: metadata.title || 'Untitled',
  date: metadata.date || '',
  description: metadata.description || '',
  content: parsedContent
};
}

// 모든 포스팅 목록 가져오기 (날짜 최신순)
export function getAllPosts(): Post[] {
  // 디렉토리가 없으면 빈 배열 반환
  if (!fs.existsSync(postsDirectory)) {
    return [];
  }

```

```

const fileNames = fs.readdirSync(postsDirectory);
const allPostsData = fileNames
  .filter(fileName => fileName.endsWith('.md'))
  .map(fileName => {
    const slug = fileName.replace(/\.md$/, '');
    const fullPath = path.join(postsDirectory, fileName);
    const fileContents = fs.readFileSync(fullPath, 'utf8');
    return parseMarkdown(fileContents, slug);
  });

return allPostsData.sort((a, b) => (a.date < b.date ? 1 : -1));
}

// 특정 포스팅 내용 가져오기
export function getPostBySlug(slug: string): Post | null {
  try {
    const fullPath = path.join(postsDirectory, `${slug}.md`);
    if (!fs.existsSync(fullPath)) return null;
    const fileContents = fs.readFileSync(fullPath, 'utf8');
    return parseMarkdown(fileContents, slug);
  } catch (e) {
    return null;
  }
}

```

7. 실제 사례 연구: 정적 웹 호스팅의 이점

마크다운 기술 블로그는 왜 데이터베이스를 사용하지 않고 SSG로 빌드하여 배포할까요?

- **비용 절감:** 서버(Node.js 인스턴스)가 계속 구동될 필요 없이 빌드된 HTML 파일만 클라우드 CDN(Vercel, AWS S3)에 얹어주면 되므로 트래픽이 몰려도 비용이 거의 0원입니다.
- **철통 보안:** 해커가 SQL Injection 등의 데이터베이스 침입 공격을 날릴 루트 자체가 차단됩니다. (서버 사이드 로직 및 DB가 없기 때문입니다.)

8. 결론 및 활용

- **요약:** SSG는 빌드 시점에 서버의 리소스를 조합해 완전무결한 HTML을 미리 생성하여 극강의 접근 속도를 보장합니다. `generateStaticParams` 는 정적 세그먼트 주소를 자동 빌드해 주는 아주 유용한 도구입니다.
- **숙제:** `posts/` 디렉토리에 나만의 마크다운 파일(예: `git-guide.md`)을 작성하고 `npm run build` 를 실행하여 새로운 정적 페이지가 자동으로 빌드되어 들어갔는지 실습해 봅니다.

9. 강사 첨언 가이드 (Instructor Notes)

💡 TIP

교안 핵심 설명 강조 사항

- **00m ~ 30m:** 라면(SSR)과 급식(SSG) 비유를 사용해 웹 속도의 차이가 가져오는 비즈니스 효과(SEO 점수 상승 및 서버 비용 감소)를 반드시 열정적으로 설명하십시오.
- **30m ~ 60m:** Node.js `fs` 모듈의 동작 원리(서버 환경에서만 돌고 클라이언트 빌드에선 지워짐)와 Frontmatter의 파싱 코드를 실습하며 문자열 제어 구조를 교육해 주세요.
- **60m ~ 120m:** 빌드 명령어(`npm run build`)를 직접 실행하여 터미널 창에 동그라미 기호 `•` (SSG) 로 생성되는 블로그 슬러그 리스트를 육안으로 확인시키는 실습을 진행하십시오.

🔥 CAUTION

학생들이 직면하는 함정 및 디버깅 팁

- 1. dangerouslySetInnerHTML 보안 주의:** 리액트에서 가공되지 않은 마크다운 변환 HTML을 강제로 바인딩할 때 쓰는 속성입니다. 외부의 입력을 그대로 렌더링하면 XSS(크로스 사이트 스크립팅) 공격 위험이 있음을 가이드해 주시고, 이번 실습처럼 내가 검증한 로컬 마크다운 파일만 쓸 때만 제한적으로 활용해야 함을 반드시 경고하세요.
- 2. 마크다운 Frontmatter 파싱 오류:** `---` 갯수가 모자라거나 YAML 포맷이 흐트러지면 본문 전체가 Frontmatter로 넘어가서 404 에러가 나거나 화면이 박살 납니다. 마크다운의 공백 문자, 줄 바꿈 기호를 꼼꼼히 확인하도록 디버깅 시 지도해 주세요.